SNOWFLAKE DATA INGESTION PROJECT

Project: E-commerce sales CSV ingestion using Snowflake

Source file: sales.csv

1. Objective

Configure a Snowflake environment, create database objects, upload a CSV file to an internal stage, load its records into a table using COPY INTO, and validate the resulting data with SQL queries.

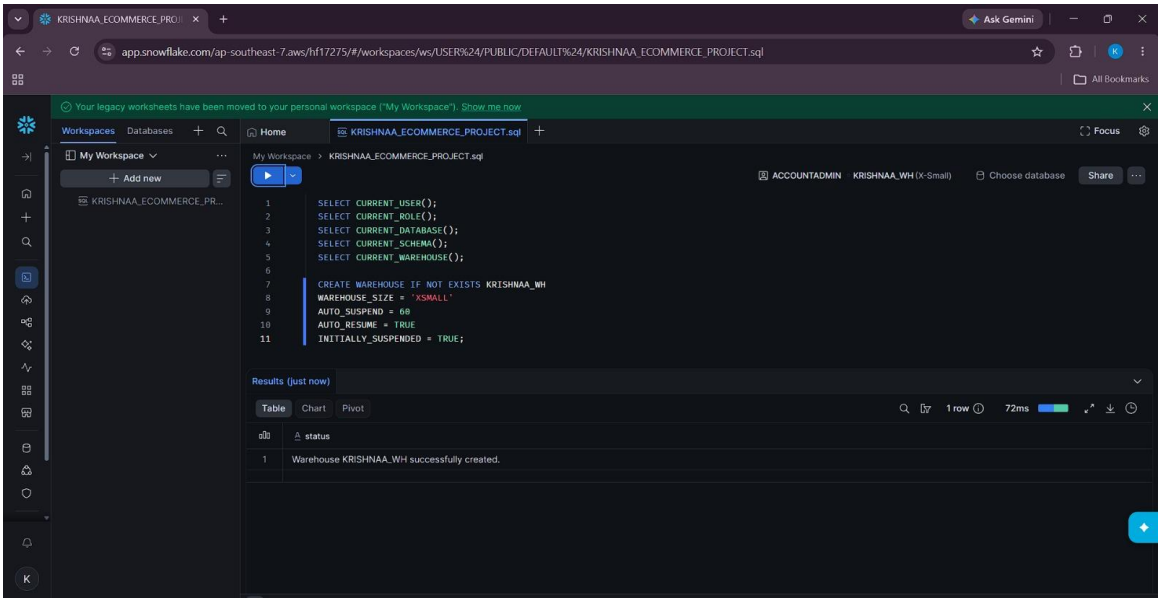
2. Environment configuration

Role	ACCOUNTADMIN
Warehouse	KRISHNAA_WH (X-SMALL)
Database	KRISHNAA_DB
Schema	FRESH
Table	SALES
File format	KRISHNAA_CSV_FORMAT
Internal stage	KRISHNAA_STAGE
SQL file	KRISHNAA_ECOMMERCE_PROJECT.sql

3. Session verification

These functions were used to identify the active Snowflake user, role, database, schema and warehouse.

```
SELECT CURRENT_USER();
SELECT CURRENT_ROLE();
SELECT CURRENT_DATABASE();
SELECT CURRENT_SCHEMA();
SELECT CURRENT_WAREHOUSE();
```

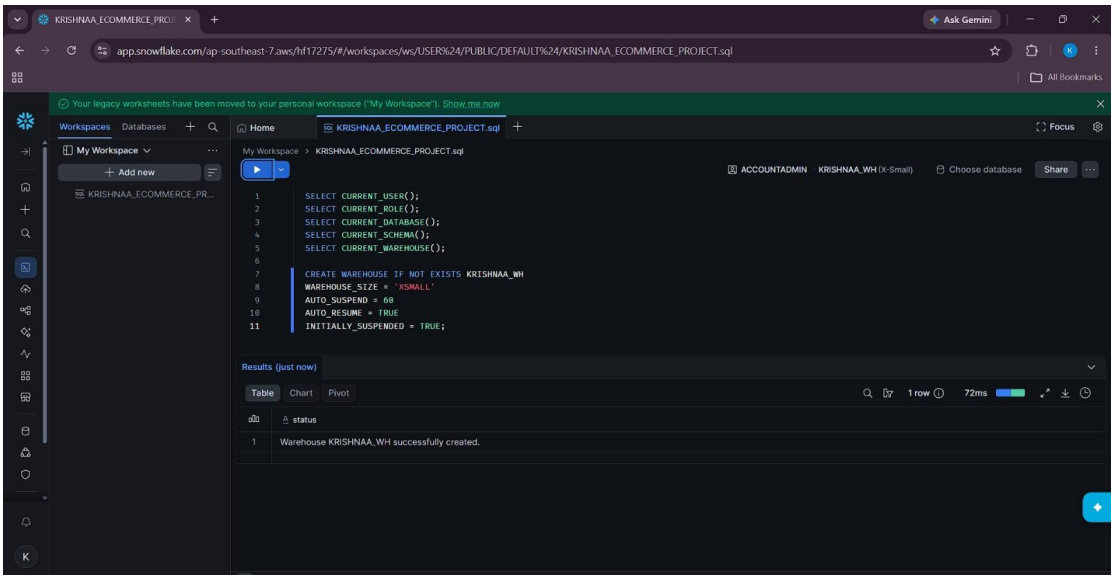


4. Create and activate the warehouse

A virtual warehouse supplies compute resources for SQL execution and data loading. The X-SMALL warehouse was configured to auto-suspend after 60 seconds and auto-resume when needed.

```
CREATE WAREHOUSE IF NOT EXISTS KRISHNAA_WH
WAREHOUSE_SIZE = 'XSMALL'
AUTO_SUSPEND = 60
AUTO_RESUME = TRUE
INITIALLY_SUSPENDED = TRUE;

SHOW WAREHOUSES LIKE 'KRISHNAA_WH';
USE WAREHOUSE KRISHNAA_WH;
SELECT CURRENT_WAREHOUSE();
```

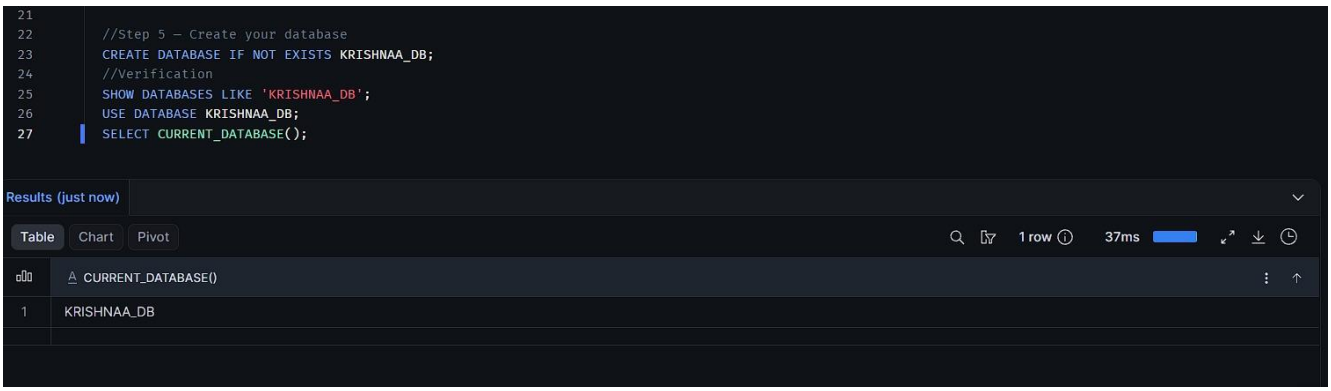


5. Create database and schema

The database stores the project objects. The FRESH schema organizes the sales table, file format and stage.

```
CREATE DATABASE IF NOT EXISTS KRISHNAA_DB;
SHOW DATABASES LIKE 'KRISHNAA_DB';
USE DATABASE KRISHNAA_DB;

CREATE SCHEMA IF NOT EXISTS FRESH;
USE SCHEMA KRISHNAA_DB.FRESH;
SHOW SCHEMAS IN DATABASE KRISHNAA_DB;
SELECT CURRENT_SCHEMA();
```



```
//Step 6 – Create your schema
CREATE SCHEMA IF NOT EXISTS FRESH;
USE SCHEMA KRISHNAA_DB.FRESH;
SHOW SCHEMAS IN DATABASE KRISHNAA_DB;
```

Results (just now)

Table | Chart | Pivot — 3 rows — 54ms

	created_on	name	is_default	is_current	database_name	owner	comment	options	reten
1	2026-11-02 02:20:08.629 -0800	FRESH	N	Y	KRISHNAA_DB	ACCOUNTADMIN			1
2	2026-11-02 02:20:29.522 -0800	INFORMATION	N	N	KRISHNAA_DB		Views describing the c		1
3	2026-11-02 02:17:50.497 -0800	PUBLIC	N	N	KRISHNAA_DB	ACCOUNTADMIN			1

6. Create the SALES table

The SALES table contains five columns matching the supplied CSV structure.

```
CREATE TABLE IF NOT EXISTS SALES (
   ORDER_ID NUMBER(10,0),
   PRODUCT VARCHAR(100),
   QUANTITY NUMBER(10,0),
   UNIT_PRICE NUMBER(12,2),
   CITY VARCHAR(100)
);

SHOW TABLES LIKE 'SALES' IN SCHEMA KRISHNAA_DB.FRESH;
DESCRIBE TABLE KRISHNAA_DB.FRESH.SALES;
```

```
//Step 7 – Create the SALES table
CREATE TABLE IF NOT EXISTS SALES (
    ORDER_ID NUMBER(10,0),
    PRODUCT VARCHAR(100),
    QUANTITY NUMBER(10,0),
    UNIT_PRICE NUMBER(12,2),
    CITY VARCHAR(100)
);
SHOW TABLES LIKE 'SALES'
IN SCHEMA KRISHNAA_DB.FRESH;

DESCRIBE TABLE KRISHNAA_DB.FRESH.SALES;
```

Results (just now)

Table | Chart | Pivot — 5 rows — 37ms

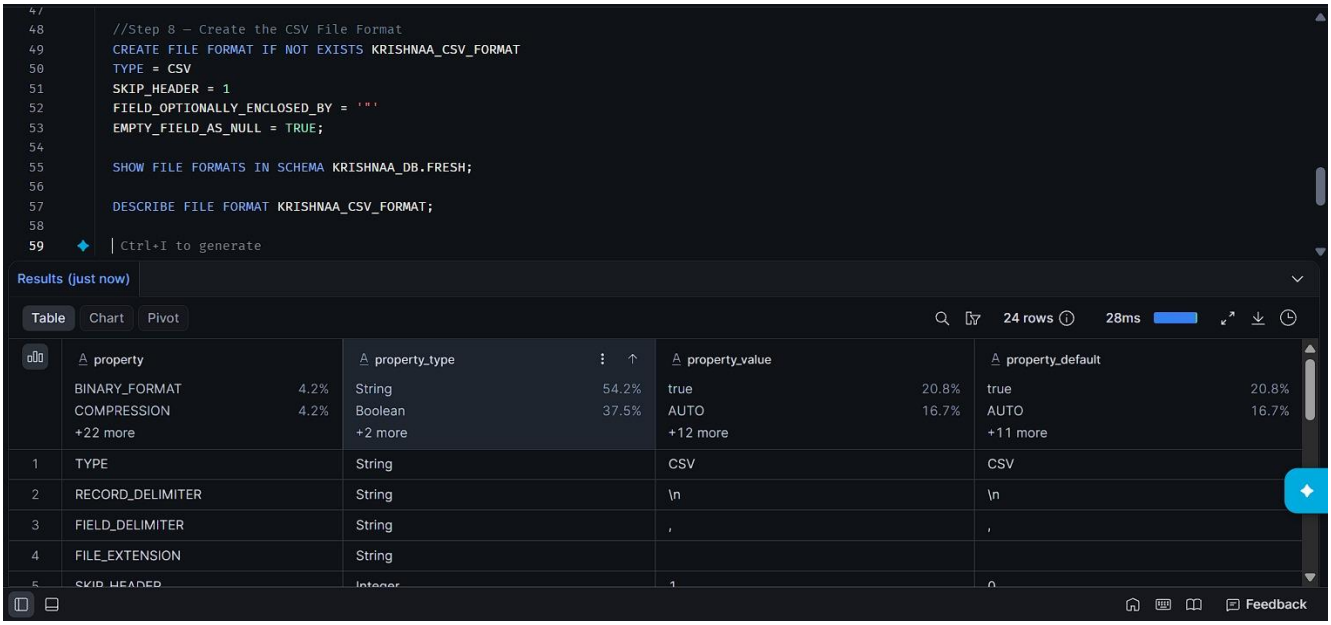
	name	type	kind	null?	default	primary key	unique key	check	expression	comme
1	ORDER_ID	NUMBER(10,0)	COLUMN	Y	null	N	N	null	null	null
2	PRODUCT	VARCHAR(100)	COLUMN	Y	null	N	N	null	null	null
3	QUANTITY	NUMBER(10,0)	COLUMN	Y	null	N	N	null	null	null
4	UNIT_PRICE	NUMBER(12,2)	COLUMN	Y	null	N	N	null	null	null
5	CITY	VARCHAR(100)	COLUMN	Y	null	N	N	null	null	null

7. Create the CSV file format

The file format tells Snowflake how to parse the CSV: skip its header row, recognize optional double-quote enclosure, and interpret empty fields as NULL.

```
CREATE FILE FORMAT IF NOT EXISTS KRISHNAA_CSV_FORMAT
TYPE = CSV
SKIP_HEADER = 1
FIELD_OPTIONALLY_ENCLOSED_BY = '"'
EMPTY_FIELD_AS_NULL = TRUE;
```

SHOW FILE FORMATS IN SCHEMA KRISHNAA_DB.FRESH;
DESCRIBE FILE FORMAT KRISHNAA_CSV_FORMAT;

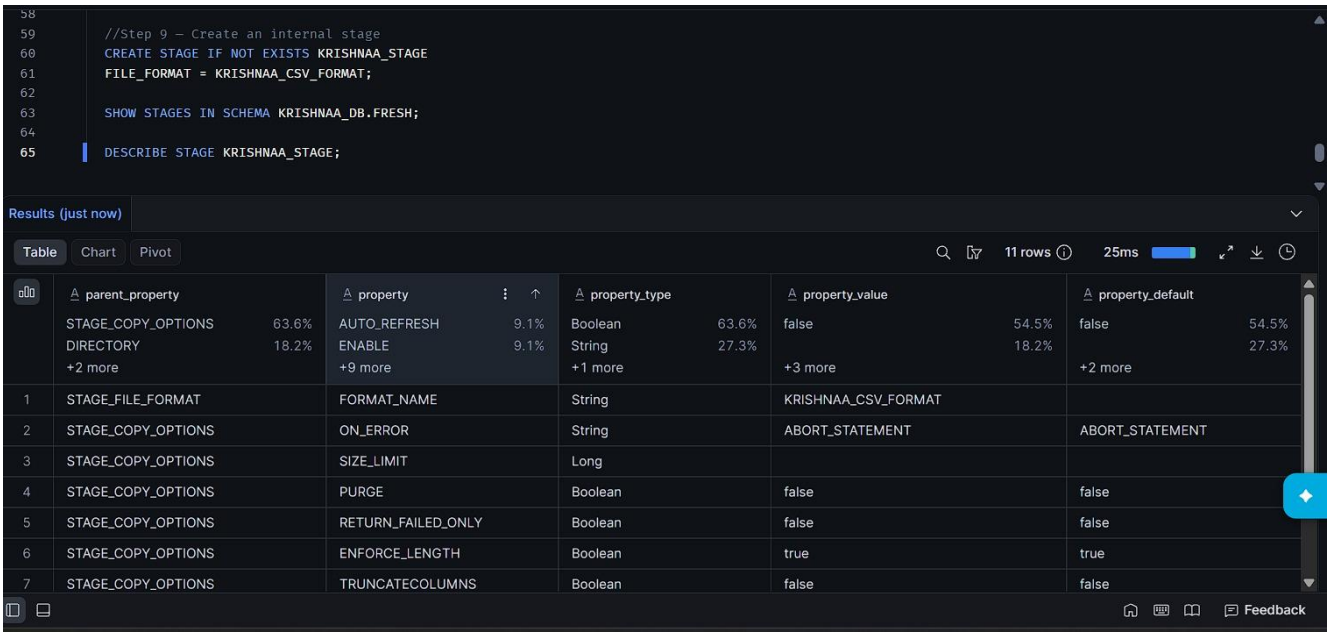


8. Create the internal stage and upload the file

An internal named stage is Snowflake-managed storage used to hold files before loading them into a table. The file sales.csv was uploaded through the Snowsight stage interface to KRISHNAA_STAGE.

CREATE STAGE IF NOT EXISTS KRISHNAA_STAGE
FILE_FORMAT = KRISHNAA_CSV_FORMAT;

SHOW STAGES IN SCHEMA KRISHNAA_DB.FRESH;
DESCRIBE STAGE KRISHNAA_STAGE;
LIST @KRISHNAA_DB.FRESH.KRISHNAA_STAGE;



9. Load data using COPY INTO

COPY INTO reads the staged CSV and inserts its records into the target SALES table. ON_ERROR = 'ABORT_STATEMENT' stops the load if an error is encountered.

```
COPY INTO KRISHNAA_DB.FRESH.SALES
FROM @KRISHNAA_DB.FRESH.KRISHNAA_STAGE
FILE_FORMAT = (
   FORMAT_NAME = 'KRISHNAA_DB.FRESH.KRISHNAA_CSV_FORMAT'
)
ON_ERROR = 'ABORT_STATEMENT';
```

```
64
65      //Step 9 — Verify CSV upload
66      LIST @KRISHNAA_DB.FRESH.KRISHNAA_STAGE;
67
68      //Step 10 — Load CSV into SALES table
69      COPY INTO KRISHNAA_DB.FRESH.SALES
70      FROM @KRISHNAA_DB.FRESH.KRISHNAA_STAGE
71      FILE_FORMAT = (
72          FORMAT_NAME = 'KRISHNAA_DB.FRESH.KRISHNAA_CSV_FORMAT'
73      )
74      ON_ERROR = 'ABORT_STATEMENT';
```

Results (just now)

Table | Chart | Pivot — 1 row — 914ms

	file	status	rows_parsed	rows_loaded	error_limit	errors_seen	first_error	first_error_line	first_error_character	
1	krishnaa_stage/sales.c	LOADED	10	10	1	0	null	null	null	null

10. Data validation and analysis

10.1 Total records

```
SELECT COUNT(*) AS TOTAL_RECORDS
FROM KRISHNAA_DB.FRESH.SALES;
```

```
76      // Step 11 - Data validation and analysis
77      //1. Count total records
78      SELECT COUNT(*) AS TOTAL_RECORDS
79      FROM KRISHNAA_DB.FRESH.SALES;
80
81      //2. Display all records
82      SELECT *
83      FROM KRISHNAA_DB.FRESH.SALES
84      ORDER BY ORDER_ID;
```

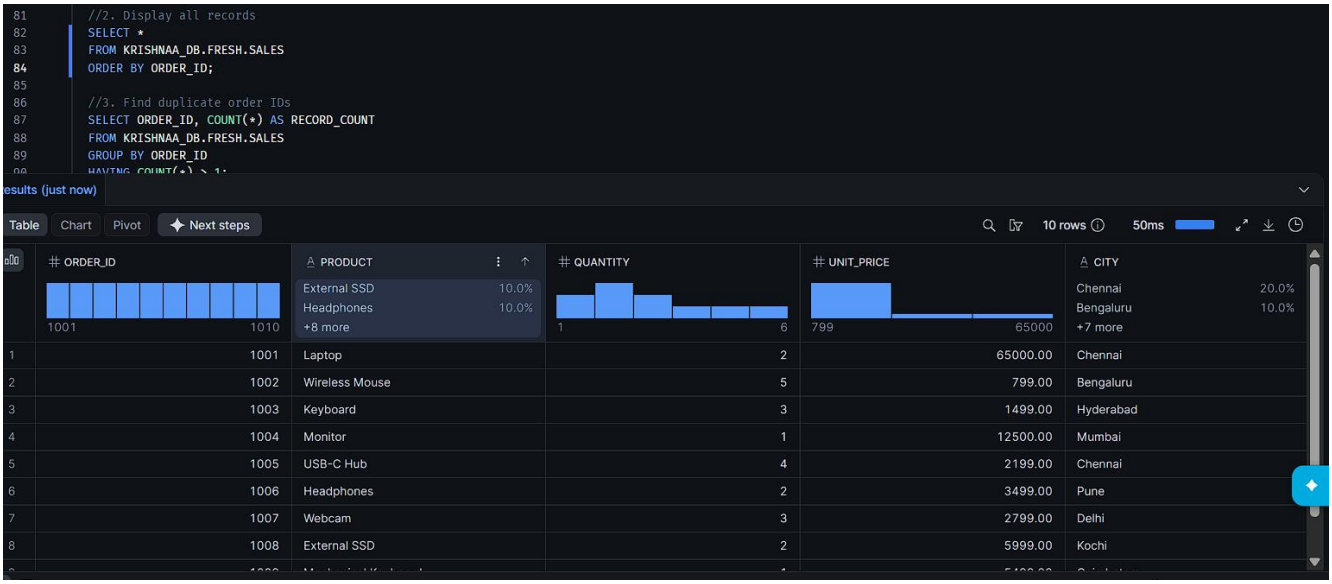
Results (just now)

Table | Chart | Pivot — 1 row — 28ms

	TOTAL_RECORDS
1	10

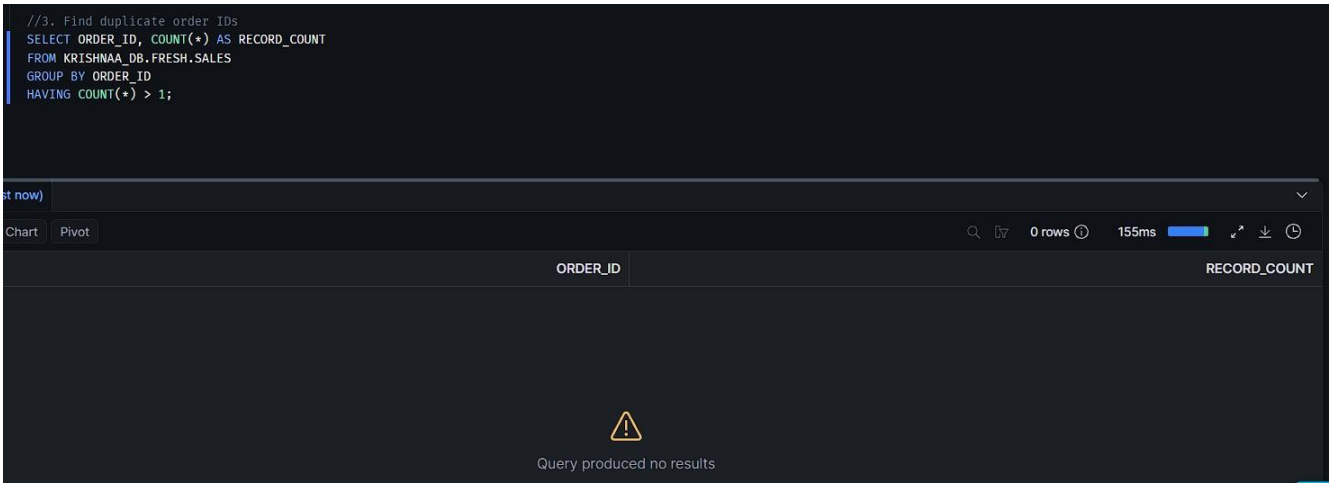
10.2 Display loaded records

```
SELECT *
FROM KRISHNAA_DB.FRESH.SALES
ORDER BY ORDER_ID;
```



10.3 Duplicate order IDs

SELECT ORDER_ID, COUNT(*) AS RECORD_COUNT
FROM KRISHNAA_DB.FRESH.SALES
GROUP BY ORDER_ID
HAVING COUNT(*) > 1;



10.4 Orders by city

SELECT CITY, COUNT(*) AS TOTAL_ORDERS
FROM KRISHNAA_DB.FRESH.SALES
GROUP BY CITY
ORDER BY TOTAL_ORDERS DESC;

```
91
92      //4. Count orders by city
93      SELECT CITY, COUNT(*) AS TOTAL_ORDERS
94      FROM KRISHNAA_DB.FRESH.SALES
95      GROUP BY CITY
96      ORDER BY TOTAL_ORDERS DESC;
```

Results (just now)

Table | Chart | Pivot — 9 rows — 52ms

	CITY	TOTAL_ORDERS
1	Chennai	2
2	Bengaluru	1
3	Hyderabad	1
4	Mumbai	1
5	Pune	1
6	Delhi	1
7	Kochi	1
8	Coimbatore	1
9	Jaipur	1

10.5 Total sales value

SELECT SUM(QUANTITY * UNIT_PRICE) AS TOTAL_SALES
FROM KRISHNAA_DB.FRESH.SALES;

```
98      //5. Calculate total sales revenue
99      SELECT SUM(QUANTITY * UNIT_PRICE) AS TOTAL_SALES
100     FROM KRISHNAA_DB.FRESH.SALES;
101
102      Ctrl+I to generate
```

Results (just now)

Table | Chart | Pivot — 1 row — 63ms

	TOTAL_SALES
1	198674.00

10.6 Quantity by product

SELECT PRODUCT, SUM(QUANTITY) AS TOTAL_QUANTITY
FROM KRISHNAA_DB.FRESH.SALES
GROUP BY PRODUCT
ORDER BY TOTAL_QUANTITY DESC;

```
102     //6. Calculate total quantity sold per product
103     SELECT PRODUCT, SUM(QUANTITY) AS TOTAL_QUANTITY
104     FROM KRISHNAA_DB.FRESH.SALES
105     GROUP BY PRODUCT
106     ORDER BY TOTAL_QUANTITY DESC;
```

Results (just now)

Table | Chart | Pivot | Next steps — 10 rows — 56ms

	PRODUCT	TOTAL_QUANTITY
	External SSD 10.0% Headphones 10.0% +8 more	1 … 6
1	Laptop Stand	6
2	Wireless Mouse	5
3	USB-C Hub	4
4	Keyboard	3
5	Webcam	3
6	Laptop	2
7	Headphones	2
8	External SSD	2